

Abstract

Keywords: Static Application Security Testing, Large Language Models,
C programming language, False Positives

Resumen

Keywords: Analizador de código estático, Modelo extenso de lenguaje,
Lenguaje de programación C, Falsos positivos

Índice

1. Introducción	7
2. Estado del arte	9
2.1. El funcionamiento de la pila de llamadas	9
2.1.1. Causas del desbordamiento de pila	11
2.2. El funcionamiento del montículo	11
2.3. Identificación de CWEs en código de pila	11
2.4. Identificación de CWEs en código de montículo	11
3. Metodología de análisis	13
4. Implementación del gran modelo de lenguaje	15
5. Resultados recopilados	17
6. Conclusiones	19

Índice de figuras

1. Estructura de memoria de un proceso C/C++. Extraída de [6] 10

1

Introducción

El lenguaje de programación C es uno de los lenguajes más populares en la industria de desarrollo de software. Este lenguaje adquiere una notable presencia en sistemas en los que se necesita del acceso a bajo nivel al hardware, la eficiencia temporal y el control explícito de los recursos. Originalmente, C se planteó como un lenguaje de programación de sistemas para la familia de sistemas operativos tipo UNIX, caracterizado técnicamente como un lenguaje liviano, de propósito general y portable [1]. Una gran cantidad de aplicaciones de alto rendimiento y sistemas críticos se implementan en C y continúan en soporte.

Sin embargo, las características que popularizaron a C también son el fondo detrás de una gestión insegura de la memoria intrínseca al lenguaje. Esta gestión insegura propicia la aparición de errores en el manejo de la memoria, que, además de ser difíciles de detectar, constituyen una de las superficies de ataque más persistentes en los lenguajes de programación *memory-unsafe*.

Los lenguajes de programación *memory-unsafe* no cuentan, por defecto, con mecanismos de prevención de errores comunes de programación relacionados con la libertad en el acceso a memoria. Bajo esta libertad, el programador gestiona manualmente la memoria, pero esta libertad conlleva la aparición de distintos tipos de vulnerabilidades, entre las que figuran los desbordamientos de pila, las filtraciones de memoria y la corrupción de memoria.

Concretamente, las vulnerabilidades de corrupción de memoria constituyen el tipo de vulnerabilidad más frecuente. La sobre-escritura de metadatos del montículo, los punteros colgantes, las lecturas de memoria no inicializada, las liberaciones inválidas de memoria o la doble liberación de memoria son debilidades reconocidas por investigadores de seguridad y cuentan con sus respectivas técnicas de protección para abordarlas [2].

Cada una de las vulnerabilidades anteriormente mencionadas se recogen en la Common Weakness Enumeration (CWE), por lo que hacemos énfasis en que las fallas que se encuentran

en programas escritos en C no son errores individuales aislados, sino que son consecuencia del diseño estructural del lenguaje.

A la vista de este escenario, la detección de debilidades supone una necesidad con objeto de asegurar la calidad y diseñar software seguro. La literatura sobre el análisis de programas señala una reducción en el coste de corrección de vulnerabilidades si se detectan tempranamente. De hecho, facilita la implementación de prácticas de seguridad «*shift-left*» e intervenir antes de que los defectos lleguen a la etapa de despliegue o de explotación [3].

Los analizadores de código estático son herramientas que permiten la detección de vulnerabilidades en el código fuente a partir de la detección de patrones. Son herramientas de referencia en la industria de desarrollo software, por ejemplo en las canalizaciones de integración continua, dado que permiten identificar estas vulnerabilidades en las fases tempranas del ciclo de vida del software. Por el contrario, hay estudios empíricos que muestran sus limitaciones en la precisión, cobertura, explicabilidad y unas altas tasas de positivos en sus reportajes [4]. Por lo tanto, detectar estas vulnerabilidades asegurando una alta fiabilidad y utilidad sigue siendo un problema abierto.

Estado del arte

En el contexto de este trabajo profundizaremos en el análisis de vulnerabilidades relacionadas con la pila de llamadas, también llamada pila o *call stack* y el montículo, también llamado *heap*. Ambas son dos regiones de memoria relevantes en la estructura de los programas en tiempo de ejecución. Tanto en C como en C++, el manejo de estas dos estructuras es explícito, dicho de otra forma, se delega en el programador la gestión de la memoria del programa [5]. Expondremos el funcionamiento del montículo y de la pila, y posteriormente explicaremos las debilidades en el lenguaje C de estos segmentos de datos para comprender como pueden explotarse.

2.1. El funcionamiento de la pila de llamadas

En la pila de llamadas se almacenan los datos relativos a las subrutinas de un programa. Cuando se realiza la llamada a una subrutina, se aloja dentro de la pila una nueva entrada denominada marco de pila, o *stack frame*, el cual contiene los parámetros de la subrutina, la dirección de retorno del contador de programa y sus variables locales.

La estrategia de gestión de datos que implementa la pila se basa en que la primera entrada que se desaloja es la última que se añadió, dicho de otra forma, la pila sigue la estrategia LIFO. Es por esto que conforme se ejecutan las subrutinas en un programa, las primeras que se alojan son las que se encuentran anidadas dentro de esta, y, consecuentemente, serán las primeras que se liberen en la pila. Es por ello que, análogamente, su funcionamiento se asemeja al de una pila de libros, apilándose estos uno encima de otro y retirándolos desde el que se encuentra más arriba al que está más abajo.

En lo referente al funcionamiento a bajo nivel, el mecanismo de alojamiento de memoria en la pila se basa en el desplazamiento de un puntero de memoria hasta el final del último marco

de pila que se insertó. Además, el lenguaje ensamblador usa instrucciones especializadas para las llamadas a subrutinas. Los compiladores de lenguajes emplean múltiples técnicas con el fin de optimizar el uso de los recursos, como las instrucciones *push* y *pop*, para introducir y sacar datos de la pila respectivamente, es un ejemplo de este tipo de optimizaciones.

Por otra parte, en el segmento de datos, la pila comparte la misma región de memoria que el montículo; con la particularidad de que los punteros de direcciones de ambos crecen en sentidos opuestos. En la mayoría de arquitecturas de computadores, se decide por convención que la pila crece decrementando el puntero mientras que el montículo lo hace incrementándolo. En la figura 1 se observa la estructura en memoria que ambas regiones comparten el mismo espacio de direcciones.

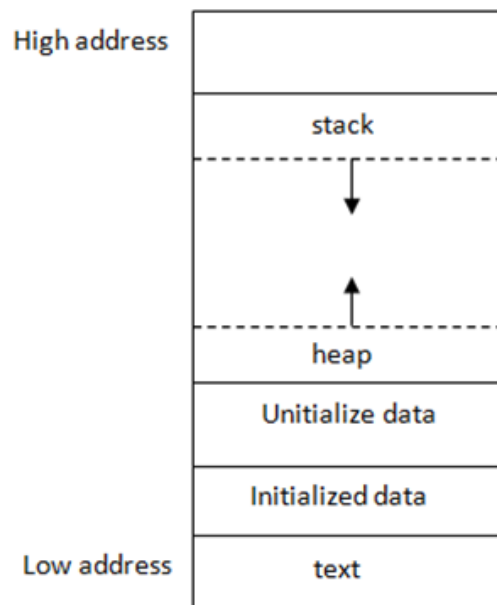


Figura 1: Estructura de memoria de un proceso C/C++. Extraída de [6]

Puesto que ambas estructuras coexisten en la misma región de memoria y crecen en sentidos opuestos, existe un límite de tamaño al cual pueden crecer la pila, el montículo en cambio no tiene esa restricción. Esta diferencia de restricciones la expondremos más a fondo en la sección dedicada al montículo. Por lo pronto, la pila de llamadas tiene un tamaño nominal específico según la plataforma de ejecución [7], dada esta restricción se desprende el error por **desbordamiento de la pila**, y su aparición se da cuando en un programa se usa más memoria de la que tiene disponible en la pila de llamadas. En otros términos, ocurre cuando el puntero de la pila de llamadas excede el tamaño límite.

2.1.1. Causas del desbordamiento de pila

Generalmente, los desbordamientos de pila pueden atribuirse a dos causas: la recursión y las variables excesivamente grandes.

En el caso de la recursión, y obviando la recursión infinita, un desbordamiento de pila puede darse si el ritmo de crecimiento de la pila se acelera conforme aumenta la recursión es más profunda, dicho de otra forma, conforme se insertan nuevos *frames*, estos contienen cada vez más datos que el anterior. Las funciones recursivas de gran profundidad pueden llevar a un desbordamiento de pila. Un ejemplo de este tipo de comportamientos puede verse en la figura ??.

Respecto a las variables excesivamente grandes, el tamaño de las variables se computa en tiempo de compilación, el desbordamiento puede darse si al calcular el tamaño de estas variables este excede el disponible en la pila.

2.2. El funcionamiento del montículo

2.3. Identificación de CWEs en código de pila

2.4. Identificación de CWEs en código de montículo

3

Metodología de análisis

4

Implementación del gran modelo de lenguaje

5

Resultados recopilados

6

Conclusiones

La conclusión

Referencias

- [1] Robert C. Seacord. *Secure coding in C and C++*. eng. 2nd ed. SEI series in software engineering Secure coding in C and C++. Place of publication not identified: Addison Wesley, 2013. ISBN: 9780132981972.
- [2] Vahid Eftekhari Moghadam et al. “Memory Integrity Techniques for Memory-Unsafe Languages: A Survey”. En: *IEEE Access* 12 (2024), págs. 43201-43221. doi: 10.1109/ACCESS.2024.3380478.
- [3] Cristina Cifuentes et al. “The role of program analysis in security vulnerability detection: Then and now”. En: *Computers & Security* 135 (2023), pág. 103463. ISSN: 0167-4048. doi: <https://doi.org/10.1016/j.cose.2023.103463>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404823003735>.
- [4] Bushra Aloraini et al. “An empirical study of security warnings from static application security testing tools”. En: *Journal of Systems and Software* 158 (2019), pág. 110427. ISSN: 0164-1212. doi: <https://doi.org/10.1016/j.jss.2019.110427>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121219302018>.
- [5] Matthew Hertz y Emery D Berger. “Quantifying the performance of garbage collection vs. explicit memory management”. En: *Proceedings of the 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 2005, págs. 313-326.
- [6] Ruohao Guo Jenny Chen. *CS 225 | Stack and Heap Memory*. URL: <https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/>.
- [7] Avd. *C/C++ maximum stack size of program on mainstream OSes*. 2009. URL: <https://stackoverflow.com/questions/1825964/c-c-maximum-stack-size-of-program-on-mainstream-oses>.